

Cart Query Optimization

Technical Documentation • Reducing payload sizes by implementing tiered data fetching

 Objective

Optimize the `getV3Cart` GraphQL query that runs on almost every page by implementing a tiered fetching strategy. Fetch only the necessary data based on the current use case.

Performance Impact

~97%

Page Load Reduction

Safe

WebSocket (Full Data)

Instant

Mini-Cart (after first fetch)

Three-Tier Data Strategy

Tier	Query	Use Case	Payload Size
Light	<code>getCartSummary</code>	Badge count, "In Cart" checks	~200-500 bytes
Medium	<code>getCartMini</code>	Mini-cart dropdown (on-demand)	~2-5 KB
Full	<code>getV3Cart</code>	Cart page, Checkout, Mutations	~15-50 KB

Data Flow Architecture

1 Page Load (Initial)

- 1 **App Bootstrap** → `initCart()`
Application initializes and triggers cart data fetch
- 2 **Fetch** `getCartSummary` (~200-500 bytes)
Lightweight query returns: quantity, itemsInCart, subscriptionId
- 3 **Update Store** → **Display Badge**
Cart badge shows item count, "Add to Cart" buttons know cart state

2 User Interactions (Mutations)

- 1 **User adds/removes item, applies coupon**
Any cart mutation is triggered by user action
- 2 **Mutation returns** `CartV3Details` (~15-50 KB)
Full cart data returned as mutation response
- 3 **Store updated with complete data**
All cart fields now available including item details

3 WebSocket Updates (Real-time)

- 1 **Subscribe to** `subscribeCartV3`
WebSocket connection established after getting subscriptionId
- 2 **Receive** `CartV3Details` (full data)
Full cart data pushed on cart changes - ensures no data loss on checkout
- 3 **Replace store with complete data**
Safe replacement with all fields including billing address, tax, etc.

4 Mini-Cart Dropdown Open

- ① **User clicks cart icon → `onMenuOpened()`**
Menu open event triggers data check
- ② **Check: `hasItemDetails?`**
Verify if `groupItemsByOwnership` already has data
- ③ **Conditional fetch or instant display**
Has details: Display instantly (0 KB)
No details: Fetch `getCartMini` (~2-5 KB)

Before vs After Comparison

✗ BEFORE

Page Load	~15-50 KB
WebSocket	~15-50 KB
Mini-Cart Open	N/A (already loaded)
Every page fetches	Full cart data

✓ AFTER

Page Load	~200-500 bytes
WebSocket	~2-5 KB
Mini-Cart Open	0 KB (if cached)
Fetches only	What's needed

Smart Conditional Fetching

The mini-cart dropdown uses intelligent caching to avoid unnecessary network requests:

```
onMenuOpened() {
  const currentCart = this.bsCartManageCoordinatorProviderService.getCurrentBsCar
  const hasItemDetails = currentCart?.groupItemsByOwnership?.length > 0

  // Only fetch if we have items but no details
  if (currentCart?.totalItems > 0 && !hasItemDetails) {
    this.bsCartManageCoordinatorProviderService.dispatchCartMini()
  }
}
```

Scenario	totalItems	hasItemDetails	Action
Empty cart	0	false	✓ No fetch (nothing to show)
Items + data from mutation/WS	> 0	true	✓ No fetch (use cached)
Items but first open	> 0	false	➔ Fetch getCartMini

GraphQL Fragments

CartSummaryDetails (Lightweight)

```
fragment CartSummaryDetails on Cart {  
  quantity  
  subscriptionId  
  sessionId  
  processId  
  groups {  
    items {  
      itemId  
      contractId  
    }  
  }  
}
```

CartMiniDetails (Medium)

```
fragment CartMiniDetails on Cart {  
  quantity  
  subscriptionId  
  sessionId  
  processId  
  net { amount, currency }  
  serviceFee { amount, currency }  
  event {  
    isInternalEvent  
    type  
    message  
    bulkDiscountRecommendation { message }  
  }  
  groups {  
    seller { memberId }  
    items {  
      itemId, memberId, contractId, itemType, free, systemDiscounts  
      price { amount, currency }  
      discount { amount, currency }  
      salePrice { amount, currency }  
      details {  
        relativeUri, title, subTitle  
        artwork { sizes { small } }  
      }  
    }  
  }  
}
```

```
}  
}
```

Key Benefits

- **Faster Initial Page Load:** ~97% reduction in cart data payload on every page
- **Safe WebSocket Updates:** Full data ensures no data loss on checkout/cart pages
- **Instant Mini-Cart:** Opens immediately when data is already cached from mutations
- **Better UX:** Pages load faster, interactions feel more responsive
- **Server Load Reduction:** Fewer large queries hitting the backend
- **Smart Caching:** Conditional fetching prevents redundant requests

Files Modified

File	Changes
<code>cart-v3.graphql</code>	Added CartSummaryDetails, CartMiniDetails fragments and queries
<code>cart-v3.factory.ts</code>	Added createCartSummary, createCartMini factory methods
<code>bs-cart-service.interface.ts</code>	Added fetchCartSummary, fetchCartMini methods
<code>cart-v3.service.ts</code>	Implemented new fetch methods with appropriate cache policies
<code>bs-cart.coordinator.ts</code>	Added dispatchCartSummary, dispatchCartMini with mergeCart pattern
<code>bs-cart-manage-coordinator-provider.service.ts</code>	Changed initCart to use dispatchCartSummary
<code>bs-mini-cart-v3-menu.component.ts</code>	Smart conditional fetch on menu open

QA Testing Checklist

Use this checklist to verify all cart functionality works correctly after the optimization changes.



Pre-Test Setup

- Open browser DevTools → Network tab
- Filter by "graphql" to monitor cart requests
- Clear cart before starting tests (if needed)
- Have test products ready to add to cart

1. Cart Badge & Initial Load

✓	Test Case	Expected Result
☐	Load any page (home, search, product)	Only <code>getCartSummary</code> request fires (small payload ~200-500B)
☐	Cart badge displays correct count	Badge shows accurate number of items
☐	Navigate between pages	No additional cart requests (data cached)
☐	Refresh page with items in cart	Badge count persists correctly

2. "In Cart" Button State

✓	Test Case	Expected Result
☐	View product already in cart	"Add to Cart" button shows "In Cart" state
☐	View product NOT in cart	"Add to Cart" button is active/clickable

✓	Test Case	Expected Result
☐	Check across different product types	Tracks, Sound Kits, Services all work correctly

3. Mini-Cart Dropdown

✓	Test Case	Expected Result
☐	Click cart icon (first time after page load)	<code>getCartMini</code> request fires, dropdown shows items with images/titles/prices
☐	Close and reopen mini-cart	NO additional request (data already loaded)
☐	Mini-cart shows item artwork	Images load correctly
☐	Mini-cart shows item titles	Titles display correctly
☐	Mini-cart shows item prices	Prices display with correct currency
☐	Mini-cart shows total	Total amount is accurate
☐	Click "Checkout" button	Navigates to checkout page
☐	Empty cart - open mini-cart	Shows empty state message, no errors

4. Add Item to Cart

✓	Test Case	Expected Result
☐	Add item from product page	Item added, badge updates, success message shows
☐	Mini-cart auto-opens after add	Dropdown opens showing newly added item
☐	Add item, then open mini-cart	NO additional request (mutation returned full data)
☐	Add multiple items	All items appear, totals update correctly

5. Remove Item from Cart

✓	Test Case	Expected Result
☐	Remove item from mini-cart	Item removed, badge updates, total recalculated
☐	Remove item from cart page	Item removed, UI updates correctly
☐	Remove last item	Cart shows empty state, badge disappears

6. Apply/Remove Coupon

✓	Test Case	Expected Result
☐	Apply valid coupon	Discount applied, totals update
☐	Apply invalid coupon	Error message displays
☐	After coupon, open mini-cart	Discounted prices shown correctly

7. Cart Page (Full Data)

✓	Test Case	Expected Result
☐	Navigate to cart page	<code>getV3Cart</code> request fires (full data needed)
☐	All item details display	Images, titles, licenses, prices all visible
☐	Billing address section works	Can view/edit billing address
☐	Tax calculations display	Tax amount shown if applicable
☐	Service fee displays	Processing fee shown correctly

8. Checkout Flow

✓	Test Case	Expected Result
☐	Proceed to checkout	All cart data available for payment
☐	Complete purchase	Transaction succeeds, cart clears
☐	After purchase, cart badge	Shows 0 or disappears

9. WebSocket Real-time Updates

✓	Test Case	Expected Result
☐	Open site in 2 browser tabs	Both tabs show same cart state
☐	Add item in Tab 1	Tab 2 badge updates via WebSocket
☐	Remove item in Tab 1	Tab 2 badge updates via WebSocket
☐	Open mini-cart in Tab 2 after WS update	Shows correct items (no extra fetch if WS provided data)

10. Edge Cases

✓	Test Case	Expected Result
☐	Shared cart URL loading	Cart loads correctly from shared link
☐	Cart with 50 items (max limit)	All items display, performance acceptable
☐	Mixed item types in cart	Tracks, Sound Kits, Services all display correctly
☐	Bulk discount activation	Discount message displays, prices update
☐	Network error during cart fetch	Graceful error handling, no crashes
☐	Slow network conditions	UI remains responsive

Pass Criteria

- All checkboxes above pass
- No JavaScript console errors related to cart
- Network requests match expected patterns (summary on load, mini on first dropdown, full on cart page)
- No regression in existing cart functionality

Generated for BeatStars Web Team • Cart Query Optimization Project